

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE

Corso di Laurea Magistrale in Ingegneria Informatica



ELABORATO DI NETWORKS AND CLOUD INFRASTRUCTURES: PROJECT WORK SLICING

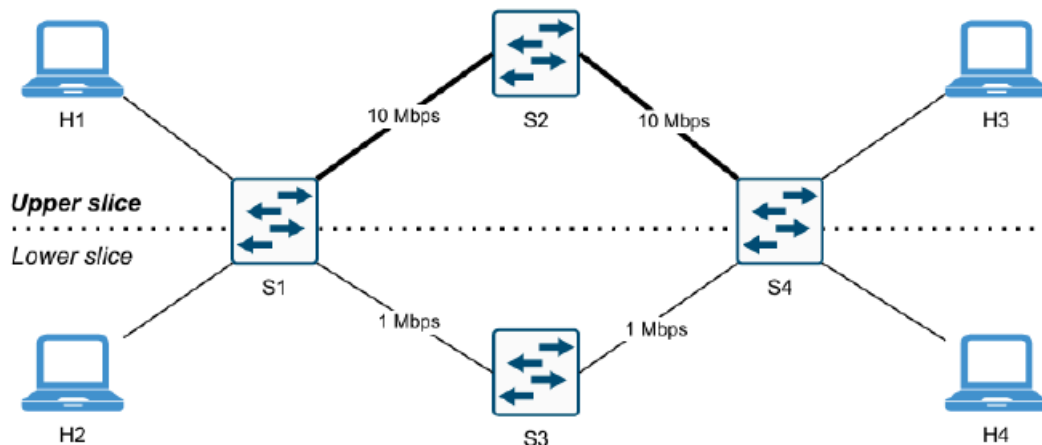
a.a. 2025-26

Studenti:

Roscigno Andrea M63001778

Topologia della Rete

La rete implementata rispecchia quanto richiesto dalla traccia del problema: la parte superiore della rete (upper slice) è caratterizzata da una larghezza di banda maggiore rispetto a quella inferiore (lower slice), con una struttura topologicamente circolare che collega i quattro switch tramite due percorsi paralleli.



La rete è composta da quattro switch OpenFlow (s1, s2, s3, s4) e quattro host (h1, h2, h3, h4), con indirizzi IP e MAC assegnati staticamente. Gli switch sono controllati da un controller remoto Ryu, che sfrutta il protocollo OpenFlow 1.3 per installare le regole di instradamento necessarie a garantire lo slicing.

La topologia è definita tramite Mininet nella classe SliceTopo, che estende Topo e specifica esplicitamente, tramite i parametri port1 e port2 di addLink, la porta fisica assegnata a ciascun capo di ogni collegamento. Questa scelta è stata fatta per garantire in maniera deterministica la corrispondenza tra la numerazione delle porte assunta dal controller e quella effettivamente utilizzata dagli switch in fase di emulazione, evitando che sia Mininet ad assegnarle automaticamente in ordine di chiamata.

```
class SliceTopo(Topo):
    def build(self):
        s1 = self.addSwitch('s1')
        s2 = self.addSwitch('s2')
        s3 = self.addSwitch('s3')
        s4 = self.addSwitch('s4')

        h1 = self.addHost('h1', ip='10.0.0.1/24', mac='00:00:00:00:00:01')
        h2 = self.addHost('h2', ip='10.0.0.2/24', mac='00:00:00:00:00:02')
        h3 = self.addHost('h3', ip='10.0.0.3/24', mac='00:00:00:00:00:03')
        h4 = self.addHost('h4', ip='10.0.0.4/24', mac='00:00:00:00:00:04')

        # Link Host -> Switch
        self.addLink(h1, s1, port1=0, port2=1)
        self.addLink(h2, s1, port1=0, port2=2)
        self.addLink(h3, s4, port1=0, port2=3)
        self.addLink(h4, s4, port1=0, port2=4)

        # Link Inter-switch
        self.addLink(s1, s2, port1=3, port2=1, bw=10)
        self.addLink(s1, s3, port1=4, port2=1, bw=1)
        self.addLink(s2, s4, port1=2, port2=1, bw=10)
        self.addLink(s3, s4, port1=2, port2=2, bw=1)
```

Di seguito si riepilogano le porte di ogni switch e i corrispondenti dispositivi collegati:

- **Switch s1:**
 - porta 1 → h1
 - porta 2 → h2
 - porta 3 → s2
 - porta 4 → s3
- **Switch s2 e s3:**
 - porta 1 → s1
 - porta 2 → s4
- **Switch s4:**
 - porta 1 → s2
 - porta 2 → s3
 - porta 3 → h3
 - porta 4 → h4

I link tra s1-s2 e s2-s4 (upper slice) hanno una banda di 10 Mbps, mentre i link tra s1-s3 e s3-s4 (lower slice) hanno una banda di 1 Mbps, in modo da rendere effettivamente distinguibili le due slice anche a livello di prestazioni di rete.

```
def run():
    topo = SliceTopo()
    ovs13 = partial(OVSSwitch, protocols='OpenFlow13')

    net = Mininet(
        topo=topo,
        controller=lambda name: RemoteController(name, ip='127.0.0.1', port=6653),
        switch=ovs13,
        link=TCLink,
        autoSetMacs=False
    )

    info('*** Avvio rete\n')
    net.start()
    info('*** Test con ping all\n')
    net.pingAll()
    info('*** Avvio CLI\n')
    CLI(net)
    info('*** Arresto rete\n')
    net.stop()
```

All'esecuzione dello script, viene innanzitutto istanziata la topologia definita con Mininet; successivamente ogni switch viene configurato per collegarsi al controller remoto all'indirizzo 127.0.0.1 sulla porta 6653. Gli switch sono di tipo OVSSwitch con protocollo OpenFlow13, mentre i collegamenti sono di tipo TCLink, che consente di simulare in maniera realistica le caratteristiche dei link, tra cui la banda disponibile. Dopo l'avvio della rete viene eseguito automaticamente un test di raggiungibilità tramite pingAll(), seguito dall'apertura di una CLI Mininet per l'interazione manuale con la rete.

Si specifica infine che la seguente topologia è stata utilizzata per entrambi i task richiesti dalla traccia (Topology Slicing e Service Slicing).

Topology slicing

Obiettivo

L'obiettivo di questo task è garantire la comunicazione esclusiva tra h1 e h3 attraverso l'upper slice (switch s1 ↔ s2 ↔ s4) e tra h2 e h4 attraverso la lower slice (switch s1 ↔ s3 ↔ s4), impedendo qualsiasi comunicazione tra host appartenenti a slice differenti (ad esempio h1-h4 o h2-h3).

Controller

Il controller è stato implementato tramite Ryu, sfruttando il protocollo OpenFlow 1.3. L'isolamento tra le slice viene realizzato tramite un meccanismo di apprendimento dinamico degli indirizzi MAC.

Su ogni switch, alla connessione con il controller, viene installata un'unica regola di table-miss: Ogni pacchetto privo di corrispondenza in una regola più specifica viene inoltrato al controller, che decide dinamicamente come gestirlo.

Il controller mantiene due dizionari fondamentali:

- `self.mac_to_port`, che mappa per ogni switch (dpid) l'associazione tra indirizzo MAC sorgente e porta d'ingresso su cui è stato osservato, aggiornato dinamicamente ad ogni pacchetto ricevuto;
- `self.slice_ports`, che definisce, per ciascuno switch, quali porte appartengono all'upper slice e quali alla lower slice:

```
self.slice_ports = {  
    1: {'upper': [1, 3], 'lower': [2, 4]},  
    2: {'upper': [1, 2], 'lower': []},  
    3: {'upper': [], 'lower': [1, 2]},  
    4: {'upper': [1, 3], 'lower': [2, 4]}  
}
```

Ogni pacchetto inoltrato al controller viene elaborato dalla funzione `packet_in_handler`, che segue i seguenti passi:

- Filtraggio LLDP: i pacchetti di tipo LLDP, utilizzati da Mininet/OVS per la discovery della topologia, vengono scartati immediatamente e non partecipano alla logica di slicing.
- Apprendimento MAC: l'indirizzo MAC sorgente del pacchetto viene associato alla porta d'ingresso corrente in `mac_to_port[dpid]`.

- Identificazione della slice: in base al MAC sorgente, il pacchetto viene classificato come appartenente all'upper slice (se proviene da h1 o h3) o alla lower slice (se proviene da h2 o h4).
- Controllo di isolamento in ingresso: si recuperano le porte valide per quella slice su quello switch (valid_ports); se la porta di ingresso non appartiene a tale insieme, il pacchetto viene scartato, garantendo che nessun traffico proveniente da una slice possa attraversare porte non autorizzate.

Decisione di forwarding:

- se il MAC di destinazione è già noto, il pacchetto viene inoltrato sulla porta corrispondente, ma solo se anch'essa appartiene alla slice corrente (altrimenti il pacchetto viene scartato, realizzando così l'isolamento anche in uscita);
- se il MAC di destinazione non è ancora noto (tipicamente in presenza di richieste ARP), il pacchetto viene inoltrato sull'altra porta appartenente alla stessa slice, confinando di fatto il flooding entro i confini della slice di appartenenza.
- Installazione della regola reattiva: viene installata una flow entry a priorità 1, con match su porta di ingresso, MAC sorgente e MAC destinazione, in modo che i pacchetti successivi dello stesso flusso vengano gestiti direttamente dallo switch senza ulteriori interazioni con il controller.

```
def packet_in_handler(self, ev):
    msg = ev.msg
    dp = msg.datapath
    ofproto = dp.ofproto
    parser = dp.ofproto_parser
    in_port = msg.match['in_port']
    dpid = dp.id

    pkt = packet.Packet(msg.data)
    eth = pkt.get_protocols(ethernet.ethernet)[0]

    if eth.ethertype == ether_types.ETH_TYPE_LLDP:
        return

    dst = eth.dst
    src = eth.src

    # Apprendimento mac_to_port
    self.mac_to_port.setdefault(dpid, {})
    self.mac_to_port[dpid][src] = in_port

    # 1. Identificazione Slice
    if src in [self.H['h1'], self.H['h3']]:
        slice_type = 'upper'
    elif src in [self.H['h2'], self.H['h4']]:
        slice_type = 'lower'
    else:
        return
```

```

valid_ports = self.slice_ports[dpid][slice_type]

# Sicurezza: ignora se la porta d'ingresso non appartiene alla slice
if not valid_ports or in_port not in valid_ports:
    return

# 2. Decisione Routing
if dst in self.mac_to_port[dpid]:
    out_port = self.mac_to_port[dpid][dst]
    # Isolamento: se la destinazione è fuori dalla slice, scarta
    if out_port not in valid_ports:
        return
else:
    # Broadcast o destinazione sconosciuta: manda sull'altra porta della slice
    out_port = valid_ports[0] if valid_ports[1] == in_port else valid_ports[1]

actions = [parser.OFPActionOutput(out_port)]

# 3. Installa flusso e inoltra pacchetto
match = parser.OFPMatch(in_port=in_port, eth_dst=dst, eth_src=src)
self.add_flow(dp, 1, match, actions)

data = msg.data if msg.buffer_id == ofproto.OFP_NO_BUFFER else None
out = parser.OFPPacketOut(datapath=dp, buffer_id=msg.buffer_id,
                           in_port=in_port, actions=actions, data=data)
dp.send_msg(out)

```

Testing

Per prima cosa è stata verificata la raggiungibilità dei dispositivi tramite il comando pingAll di Mininet, che si prevede confermi la comunicazione esclusiva tra h1-h3 e h2-h4.

```

mininet> pingall
*** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)
mininet>

```

Successivamente, per verificare che il traffico attraversi effettivamente lo slice corretto, sono stati aperti i terminali degli switch tramite xterm dalla CLI di Mininet, utilizzando tcpdump per osservare il traffico in transito sulle porte di interesse.

Per verificare l'upper slice sono state osservate le porte s1-eth3, s2-eth1/eth2, s3-eth2 e s4-eth1 durante l'esecuzione di h1 ping -c 5 h3; la porta s3-eth2 è stata monitorata anche per verificare l'assenza di traffico su di essa, a conferma dell'isolamento.

```

"Node: s1" (root)
th 64
13:13:09,085349 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 1, length
64
13:13:10,085259 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 2, leng
th 64
13:13:10,085378 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 2, length
64
13:13:11,149770 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 3, leng
th 64
13:13:11,150150 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 3, length
64
13:13:12,172166 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 4, leng
th 64
13:13:12,172208 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 4, length
64
13:13:13,201465 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 5, leng
th 64
13:13:13,201725 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 5, length
64
13:13:14,355754 ARP, Request who-has 10.0.0.3 tell 10.0.0.1, length 28
13:13:14,355869 ARP, Request who-has 10.0.0.1 tell 10.0.0.3, length 28
13:13:14,355887 ARP, Reply 10.0.0.3 is-at 00:00:00:00:00:03, length 28
13:13:14,355912 ARP, Reply 10.0.0.1 is-at 00:00:00:00:00:01, length 28
[]

"Node: s2" (root)
th 64
13:13:09,085328 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 1, length
64
13:13:10,085272 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 2, leng
th 64
13:13:10,085374 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 2, length
64
13:13:11,149782 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 3, leng
th 64
13:13:11,150147 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 3, length
64
13:13:12,172177 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 4, leng
th 64
13:13:12,172206 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 4, length
64
13:13:13,201481 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 5, leng
th 64
13:13:13,201714 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 5, length
64
13:13:14,355837 ARP, Request who-has 10.0.0.1 tell 10.0.0.3, length 28
13:13:14,355845 ARP, Request who-has 10.0.0.3 tell 10.0.0.1, length 28
13:13:14,355886 ARP, Reply 10.0.0.3 is-at 00:00:00:00:00:03, length 28
13:13:14,355914 ARP, Reply 10.0.0.1 is-at 00:00:00:00:00:01, length 28
[]

"Node: s3" (root)
root@andrea-VirtualBox: /home/andrea/progetto/codice/2# tcpdump -i s3-eth2 -nn
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on s3-eth2, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:12:39,981009 IP6 fe80::e050:a2ff:fec5:c969 > ff02::2: ICMP6, router solicitat
ion, length 16
[]

"Node: s4" (root)
th 64
13:13:09,085327 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 1, length
64
13:13:10,085274 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 2, leng
th 64
13:13:10,085373 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 2, length
64
13:13:11,149785 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 3, leng
th 64
13:13:11,150144 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 3, length
64
13:13:12,172180 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 4, leng
th 64
13:13:12,172205 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 4, length
64
13:13:13,201484 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 32037, seq 5, leng
th 64
13:13:13,201710 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 32037, seq 5, length
64
13:13:14,355835 ARP, Request who-has 10.0.0.1 tell 10.0.0.3, length 28
13:13:14,355845 ARP, Request who-has 10.0.0.3 tell 10.0.0.1, length 28
13:13:14,355886 ARP, Reply 10.0.0.3 is-at 00:00:00:00:00:03, length 28
13:13:14,355914 ARP, Reply 10.0.0.1 is-at 00:00:00:00:00:01, length 28
[]

```

```

mininet> xterm s1 s2 s3 s4
mininet> h1 ping -c 5 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=2.67 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.189 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.440 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.097 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=0.349 ms

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4117ms
rtt min/avg/max/mdev = 0.097/0.748/2.665/0.965 ms

```

Per verificare la lower slice sono state osservate le porte s1-eth4,s2 eth-2, s3-eth1/eth2, s4-eth2 durante l'esecuzione di h2 ping -c 5 h4.


```
"Node: s1" (root)      "Node: s2" (root)
th 64
13:34:59.246400 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 1, length 64
13:35:00.265329 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 2, length 64
13:35:00.265402 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 2, length 64
13:35:01.292115 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 3, length 64
13:35:01.292176 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 3, length 64
13:35:02.329096 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 4, length 64
13:35:02.329145 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 4, length 64
13:35:03.340559 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 5, length 64
13:35:03.340613 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 5, length 64
13:35:04.492629 ARP, Request who-has 10.0.0.4 tell 10.0.0.2, length 28
13:35:04.492680 ARP, Request who-has 10.0.0.2 tell 10.0.0.4, length 28
13:35:04.492716 ARP, Reply 10.0.0.2 is-at 00:00:00:00:00:02, length 28
13:35:04.492728 ARP, Reply 10.0.0.4 is-at 00:00:00:00:00:04, length 28

"Node: s3" (root)      "Node: s4" (root)
th 64
13:34:59.246382 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 1, length 64
13:35:00.265347 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 2, length 64
13:35:00.265398 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 2, length 64
13:35:01.292129 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 3, length 64
13:35:01.292173 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 3, length 64
13:35:02.329108 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 4, length 64
13:35:02.329142 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 4, length 64
13:35:03.340572 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 5, length 64
13:35:03.340611 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 5, length 64
13:35:04.492615 ARP, Request who-has 10.0.0.2 tell 10.0.0.4, length 28
13:35:04.492686 ARP, Request who-has 10.0.0.4 tell 10.0.0.2, length 28
13:35:04.492717 ARP, Reply 10.0.0.2 is-at 00:00:00:00:00:02, length 28
13:35:04.492727 ARP, Reply 10.0.0.4 is-at 00:00:00:00:00:04, length 28

th 64
13:34:59.246380 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 1, length 64
13:35:00.265352 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 2, length 64
13:35:00.265396 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 2, length 64
13:35:01.292133 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 3, length 64
13:35:01.292172 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 3, length 64
13:35:02.329111 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 4, length 64
13:35:02.329141 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 4, length 64
13:35:03.340576 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 63726, seq 5, length 64
13:35:03.340609 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 63726, seq 5, length 64
13:35:04.492611 ARP, Request who-has 10.0.0.2 tell 10.0.0.4, length 28
13:35:04.492687 ARP, Request who-has 10.0.0.4 tell 10.0.0.2, length 28
13:35:04.492717 ARP, Reply 10.0.0.2 is-at 00:00:00:00:00:02, length 28
13:35:04.492727 ARP, Reply 10.0.0.4 is-at 00:00:00:00:00:04, length 28

mininet> h2 ping -c 5 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=2.20 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=0.332 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=0.138 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=0.118 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=0.128 ms

--- 10.0.0.4 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4094ms
rtt min/avg/max/mdev = 0.118/0.582/2.195/0.810 ms
mininet>
```

Infine, per dimostrare l'effettivo isolamento tra le slice, è stato eseguito un ping tra h1 e h4 (host appartenenti a slice differenti), verificando che la comunicazione non vada a buon fine.

```
"Node: s1" (root)
root@andrea-VirtualBox:/home/andrea/progetto/codice/2# tcpdump -i s1-eth3 -nn
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on s1-eth3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:30:14.581739 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:15.596947 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:16.639968 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:18.668331 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:19.726404 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:20.795688 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
^

"Node: s2" (root)
root@andrea-VirtualBox:/home/andrea/progetto/codice/2# tcpdump -i s2-eth2 -nn
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on s2-eth2, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:30:14.581813 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:15.596967 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:16.639979 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:18.668339 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:19.726420 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:20.795701 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
^

"Node: s3" (root)
root@andrea-VirtualBox:/home/andrea/progetto/codice/2# tcpdump -i s3-eth2 -nn
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on s3-eth2, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^

"Node: s4" (root)
root@andrea-VirtualBox:/home/andrea/progetto/codice/2# tcpdump -i s4-eth1 -nn
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on s4-eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:30:14.581815 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:15.596959 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:16.639981 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:18.668341 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:19.726423 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
13:30:20.795704 ARP, Request who-has 10.0.0.4 tell 10.0.0.1, length 28
^

mininet> h1 ping -c 5 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable
From 10.0.0.1 icmp_seq=4 Destination Host Unreachable
From 10.0.0.1 icmp_seq=5 Destination Host Unreachable

--- 10.0.0.4 ping statistics ---
5 packets transmitted, 0 received, +5 errors, 100% packet loss, time 4085ms
pipe 4
mininet> ^
```

Service slicing

Per questo secondo task la topologia della rete resta invariata rispetto a quella descritta nel capitolo precedente, ma cambia il criterio con cui la comunicazione viene ripartita tra le due slice. A differenza del Topology Slicing, in questo caso tutti i dispositivi devono poter comunicare liberamente tra loro: non è più l'identità dell'host a determinare la slice di appartenenza, bensì la tipologia del traffico generato. Il traffico video, identificato tramite pacchetti UDP diretti alla porta 9999, viene instradato sull'upper slice per sfruttarne la maggiore banda disponibile, mentre ogni altro tipo di traffico viene instradato sulla lower slice. Poiché tutti gli host sono reciprocamente raggiungibili, gli switch di bordo s1 e s4 assumono un ruolo più articolato rispetto al task precedente: non si limitano a smistare il traffico verso l'altro lato della rete secondo la slice corretta, ma devono anche gestire correttamente le comunicazioni locali, ovvero quelle tra h1 e h2 su s1 e quelle tra h3 e h4 su s4, che non necessitano di attraversare gli switch di transito.

controller

Il controller adottato per questo task mantiene un approccio a regole interamente precaricate: tutte le flow entry vengono installate in blocco al momento della connessione dello switch al controller, all'interno della funzione `switch_features_handler`, senza ricorrere ad alcun meccanismo di apprendimento dinamico degli indirizzi MAC. Questa scelta è coerente con la natura del task: la classificazione del traffico si basa su proprietà del pacchetto note a priori, quali il protocollo di trasporto e la porta di destinazione, e non richiede quindi che il controller apprenda progressivamente la posizione degli host nella rete.

Come nel task precedente, su ogni switch viene installata una regola di default che scarta ogni pacchetto privo di corrispondenza in una regola più specifica: garantisce che qualsiasi pacchetto non riconducibile a nessun'altra regola venga scartato.

Per quanto riguarda la gestione del protocollo ARP, poiché in questo scenario tutti i dispositivi devono risultare reciprocamente raggiungibili, le richieste ricevute dagli switch di bordo vengono inoltrate su tutte le porte che conducono a un host o alla lower slice, escludendo deliberatamente la porta verso la upper slice: quest'ultima è infatti riservata al solo traffico video e non deve essere attraversata da traffico di controllo.

Su s1 la regola ARP inoltra il pacchetto verso h1, h2 e s3, escludendo la porta verso s2:

PRIORITA'	MATCH	AZIONE	DESCRIZIONE
100	eth_type=0x0806 (ARP)	Output: 1 (h1), 2 (h2), 4 (s3)	Invia l'ARP agli host locali e alla lower slice, escludendo s2

Sullo switch s2 non è prevista alcuna regola dedicata all'ARP: trattandosi di uno switch di puro transito per il solo traffico video, in condizioni normali non dovrebbe mai ricevere pacchetti di questo tipo; qualora ciò accadesse comunque, il pacchetto verrebbe semplicemente scartato dalla regola di default.

Lo switch s3, agendo da transito per la lower slice e quindi anche per il traffico di controllo, effettua invece un flooding completo su tutte le porte attive diverse da quella di ingresso:

PRIORITA'	MATCH	AZIONE	DESCRIZIONE
100	eth_type=0x0806 (ARP)	FLOOD	Inoltra l'ARP su tutte le porte attive tranne quella di ingresso

Su s4 la logica è simmetrica a quella di s1: il pacchetto viene inoltrato verso h3, h4 e s3, escludendo la porta verso s2:

PRIORITA'	MATCH	AZIONE	DESCRIZIONE
100	eth_type=0x0806 (ARP)	Output: 3 (h3), 4 (h4), 2 (s3)	Invia l'ARP agli host locali e alla lower slice, escludendo s2

Per quanto riguarda l'instradamento del traffico dati, le regole installate su s1 riflettono la duplice esigenza di gestire sia il traffico locale tra h1 e h2 sia quello diretto verso l'altro lato della rete. Il traffico video scambiato direttamente tra h1 e h2 riceve la priorità più alta, in quanto rappresenta il caso più specifico e non deve mai essere inoltrato verso gli switch di transito:

PRIORITA'	MATCH	AZIONE	DESCRIZIONE
310	IP, UDP, dst_port=9999, src=h1, dst=h2	Output: h2	Traffico video locale da h1 a h2, non lascia lo switch
310	IP, UDP, dst_port=9999, src=h2, dst=h1	Output: h1	Traffico video locale da h2 a h1, non lascia lo switch

Subito sotto, a priorità 300, si trovano le regole che indirizzano verso la upper slice il traffico video proveniente da h1 o h2 e diretto verso l'altro lato della rete, insieme alle regole simmetriche che gestiscono il traffico di ritorno proveniente da s2:

PRIORITA'	MATCH	AZIONE	DESCRIZIONE
300	IP, UDP, dst_port=9999, in_port=1 (h1)	Output: 3 (s2)	Traffico video da h1 instradato sulla upper slice
300	IP, UDP, dst_port=9999, in_port=2 (h2)	Output: 3 (s2)	Traffico video da h2 instradato sulla upper slice
300	IP, in_port=3 (da s2), dst=h1	Output: 1 (h1)	Traffico di ritorno dalla upper slice verso h1
300	IP, in_port=3 (da s2), dst=h2	Output: 1 (h2)	Traffico di ritorno dalla upper slice verso h2

A priorità 210 sono collocate le regole per il traffico standard scambiato localmente tra h1 e h2, più specifiche rispetto a quelle generiche di instradamento verso l'esterno ma meno prioritarie rispetto al traffico video:

PRIORITA'	MATCH	AZIONE	DESCRIZIONE
210	IP, dst=h1	Output: h1	Traffico IP normale locale diretto a h1
210	IP, dst=h2	Output: h2	Traffico IP normale locale diretto a h2

Infine, a priorità 200, si trovano le regole più generiche, che instradano verso la lower slice tutto il restante traffico IP diretto a h3 o h4, insieme alle regole di ritorno per il traffico proveniente da s3:

PRIORITA'	MATCH	AZIONE	DESCRIZIONE
200	IP, dst=h3	Output: 4 (s3)	Traffico standard verso h3 instradato sulla lower slice
200	IP, dst=h4	Output: 4 (s3)	Traffico standard verso h4 instradato sulla lower slice
200	IP, in_port=4 (da s3), dst=h1	Output: h1	Traffico di ritorno dalla lower slice verso h1
200	IP, in_port=4 (da s3), dst=h2	Output: h2	Traffico di ritorno dalla lower slice verso h2

Gli switch s2 e s3 svolgono un ruolo di puro transito e le rispettive regole si limitano a inoltrare il traffico ricevuto sulla porta opposta a quella di ingresso, mantenendo la priorità coerente con la slice di appartenenza:

SWITCH	PRIORITA'	MATCH	AZIONE	DESCRIZIONE
s2	300	IP, in_port=1 (da s1)	Output: 2 (verso s4)	Instrada il traffico video in avanti
s2	300	IP, dst=IP, in_port=2 (da s1)	Output: 1 (verso s1)	Instrada il traffico video di

		s4)		ritorno
s3	200	IP, in_port=1 (da s1)	Output: 2 (verso s4)	Instrada il traffico standard in avanti
s3	200	IP, in_port=2 (da s4)	Output: 1 (verso s1)	Instrada il traffico standard di ritorno

Lo switch s4 replica specularmente la logica di s1, gestendo questa volta il traffico locale tra h3 e h4 e quello diretto verso h1 e h2 attraverso gli switch di transito:

PRIORITA'	MATCH	AZIONE	DESCRIZIONE
310	IP, UDP, dst_port=9999, src=h3, dst=h4	Output: h4	Traffico video locale da h3 a h4
310	IP, UDP, dst_port=9999, src=h4, dst=h3	Output: h3	Traffico video locale da h4 a h3
300	IP, UDP, dst_port=9999, in_port=3 (h3)	Output: 1 (s2)	Traffico video da h3 instradato sulla upper slice
300	IP, UDP, dst_port=9999, in_port=4 (h4)	Output: 1 (s2)	Traffico video da h4 instradato sulla upper slice
300	IP, in_port=1 (da s2), dst=h3	Output: 3 (h3)	Traffico di ritorno dalla upper slice verso h3
300	IP, in_port=1 (da s2), dst=h4	Output: 4 (h4)	Traffico di ritorno dalla upper slice verso h4
210	IP, dst=h3	Output: h3	Traffico IP normale locale diretto a h3
210	IP, dst=h4	Output: h4	Traffico IP normale locale diretto a h4
200	IP, dst=h1	Output: 2 (s3)	Traffico standard verso h1 instradato sulla lower slice
200	IP, dst=h2	Output: 2 (s3)	Traffico standard verso h2 instradato sulla lower slice

200	IP, in_port=2 (da s3), dst=h3	Output: h3	Traffico di ritorno dalla lower slice verso h3
200	IP, in_port=2 (da s3), dst=h4	Output: h4	Traffico di ritorno dalla lower slice verso h4

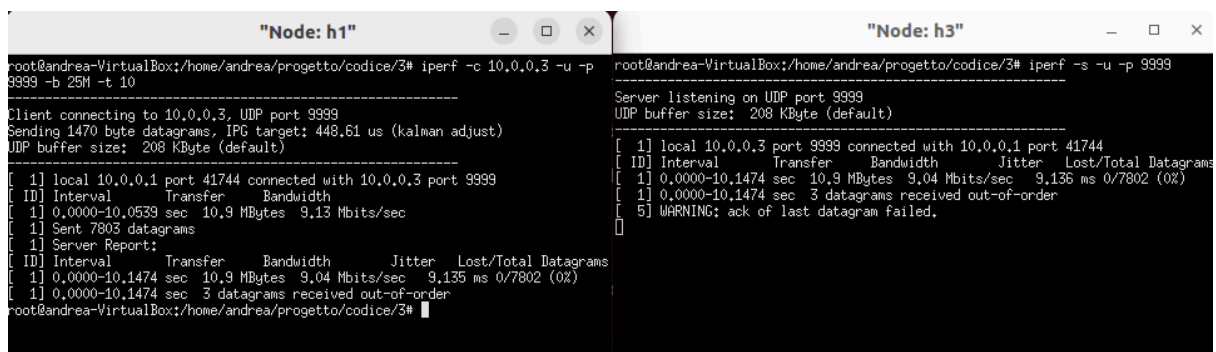
L'organizzazione delle priorità riflette una gerarchia di specificità decrescente. Il traffico video scambiato localmente riceve la priorità massima poiché rappresenta il caso più circoscritto; segue il traffico video destinato all'altro lato della rete, che deve prevalere sul traffico generico proprio per evitare che quest'ultimo, essendo catturato da un match molto ampio, lo trasferisca erroneamente sulla lower slice. Il traffico standard locale riceve a sua volta una priorità maggiore rispetto al traffico standard generico, in quanto anch'esso rappresenta un caso più specifico rispetto all'instradamento verso s2 o s3. Per quanto riguarda le strutture dati impiegate, il controller riutilizza il dizionario self.H già introdotto nel Topology Slicing e introduce una nuova struttura, self.PORT_MAP, per la mappatura nome-nodo→porta di ciascuno switch. .

testing

Il primo test eseguito verifica la piena raggiungibilità di tutti i dispositivi tramite il comando pingAll, che in questo scenario deve restituire una copertura completa, a differenza di quanto osservato per il Topology Slicing.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
```

Per verificare che la classificazione del traffico avvenga correttamente si è ricorsi allo strumento iperf, che consente di generare traffico TCP o UDP e di misurarne banda, jitter e perdita di pacchetti. Sono stati condotti quattro test distinti. Nel primo, una trasmissione video UDP sulla porta 9999 tra h1 e h3 ha fatto registrare una banda di circa 10 Mbit/s su entrambe le direzioni, valore coerente con l'attraversamento della upper slice.

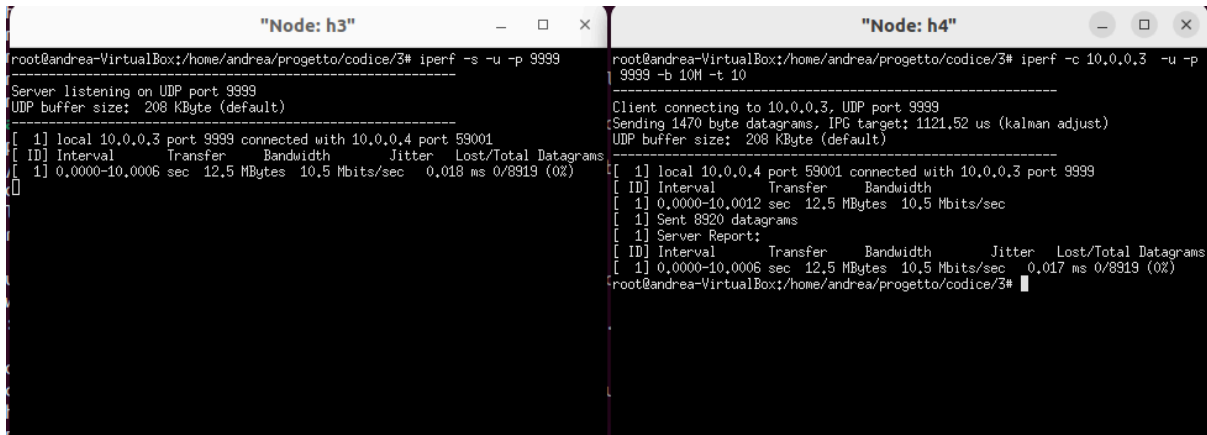


```

"Node: h1"
root@andrea-VirtualBox: /home/andrea/progetto/codice/3# iperf -c 10.0.0.3 -u -p 9999 -b 25M -t 10
Client connecting to 10.0.0.3, UDP port 9999
Sending 1470 byte datagrams, IPG target: 448.61 us (kalman adjust)
UDP buffer size: 208 KByte (default)
[ 1] local 10.0.0.1 port 41744 connected with 10.0.0.3 port 9999
[ ID] Interval      Transfer      Bandwidth      Jitter  Lost/Total Datagrams
[ 1] 0.0000-10.0539 sec  10.9 MBytes  9.13 Mbits/sec  9.135 ms  0/7802 (0%)
[ 1] Sent 7803 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter  Lost/Total Datagrams
[ 1] 0.0000-10.1474 sec  10.9 MBytes  9.04 Mbits/sec  9.135 ms  0/7802 (0%)
[ 1] 0.0000-10.1474 sec  3 datagrams received out-of-order
root@andrea-VirtualBox: /home/andrea/progetto/codice/3#

"Node: h3"
root@andrea-VirtualBox: /home/andrea/progetto/codice/3# iperf -s -u -p 9999
Server listening on UDP port 9999
UDP buffer size: 208 KByte (default)
[ 1] local 10.0.0.3 port 9999 connected with 10.0.0.1 port 41744
[ ID] Interval      Transfer      Bandwidth      Jitter  Lost/Total Datagrams
[ 1] 0.0000-10.1474 sec  10.9 MBytes  9.04 Mbits/sec  9.135 ms  0/7802 (0%)
[ 1] 0.0000-10.1474 sec  3 datagrams received out-of-order
[ 5] WARNING: ack of last datagram failed.
```

Nel secondo test, una trasmissione video tra h3 e h4 tramite s4 ha mostrato una banda poco superiore a 10 Mbit/s, confermando anch'essa il transito sulla upper slice.



The image shows two terminal windows side-by-side. The left window is titled "Node: h3" and the right is titled "Node: h4". Both show the execution of the 'iperf' command for a UDP test. In Node h3, the server is listening on port 9999. In Node h4, the client is connecting to 10.0.0.3 on port 9999. The results show a bandwidth of approximately 10.5 Mbits/sec, which is slightly above the 10 Mbit/s target.

```
root@andrea-VirtualBox: /home/andrea/progetto/codice/3# iperf -s -u -p 9999
Server listening on UDP port 9999
UDP buffer size: 208 KByte (default)

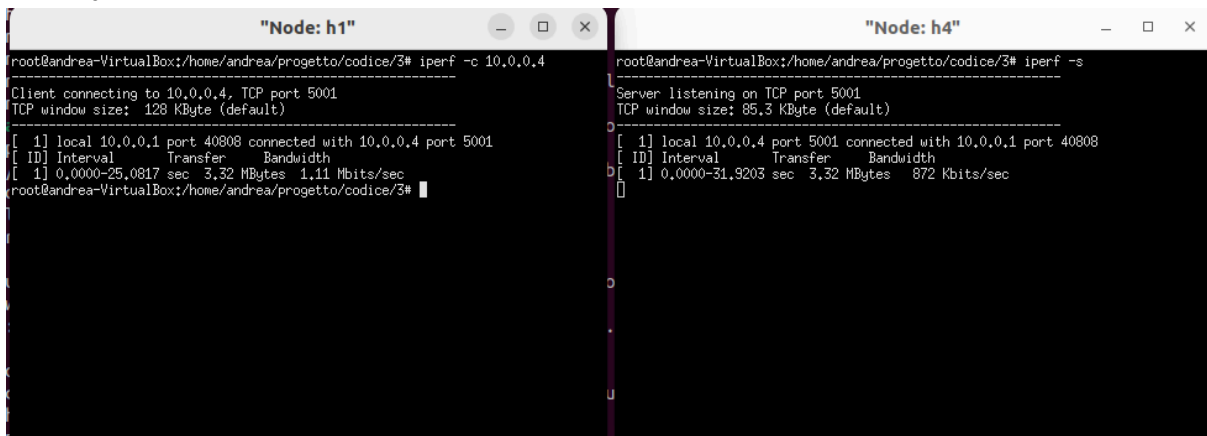
[ 1] local 10.0.0.3 port 9999 connected with 10.0.0.4 port 59001
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-10.0006 sec 12.5 MBytes  10.5 Mbits/sec  0.018 ms  0/8919 (0%)

root@andrea-VirtualBox: /home/andrea/progetto/codice/3# iperf -c 10.0.0.3 -u -p 9999 -b 10M -t 10
Client connecting to 10.0.0.3, UDP port 9999
Sending 1470 byte datagrams, IPG target: 1121.52 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 10.0.0.4 port 59001 connected with 10.0.0.3 port 9999
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-10.0012 sec 12.5 MBytes  10.5 Mbits/sec
[ 1] Sent 8920 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-10.0006 sec 12.5 MBytes  10.5 Mbits/sec  0.017 ms  0/8919 (0%)

root@andrea-VirtualBox: /home/andrea/progetto/codice/3#
```

Nel terzo test è stata avviata una trasmissione standard TCP, non riconosciuta come traffico video, tra h1 e h4 sulla porta 5000, con una banda richiesta di 10 Mbit/s: la banda effettivamente misurata è risultata pari a circa 1.11 Mbit/s, valore coerente con la capacità della lower slice, accompagnato da lievi segnali di congestione (mancata ricezione di alcuni ACK e jitter elevato) dovuti alla banda ridotta del canale.



The image shows two terminal windows side-by-side. The left window is titled "Node: h1" and the right is titled "Node: h4". Both show the execution of the 'iperf' command for a TCP test. In Node h1, the client is connecting to 10.0.0.4 on port 5001. In Node h4, the server is listening on port 5001. The results show a bandwidth of approximately 872 Kbits/sec, which is significantly lower than the requested 10 Mbit/s, indicating congestion on the lower slice.

```
root@andrea-VirtualBox: /home/andrea/progetto/codice/3# iperf -c 10.0.0.4
Client connecting to 10.0.0.4, TCP port 5001
TCP window size: 128 KByte (default)

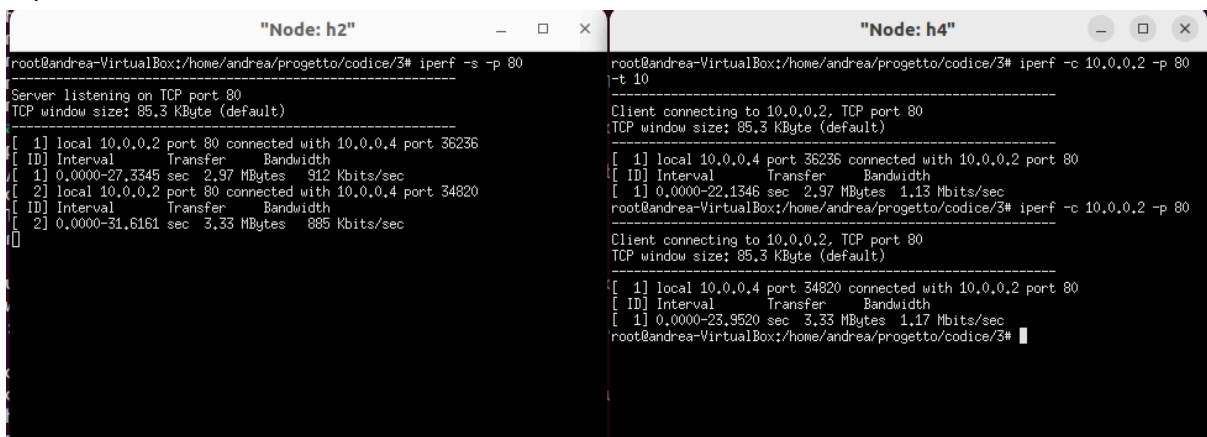
[ 1] local 10.0.0.1 port 40808 connected with 10.0.0.4 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-25.0817 sec 3.32 MBytes  1.11 Mbits/sec

root@andrea-VirtualBox: /home/andrea/progetto/codice/3# iperf -s
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)

[ 1] local 10.0.0.4 port 5001 connected with 10.0.0.1 port 40808
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-31.9203 sec 3.32 MBytes  872 Kbits/sec

root@andrea-VirtualBox: /home/andrea/progetto/codice/3#
```

Nel quarto test, una trasmissione TCP tra h4 e h2 ha fatto registrare una banda di poco superiore a 1 Mbit/s, anch'essa coerente con l'instradamento sulla lower slice.



The image shows two terminal windows side-by-side. The left window is titled "Node: h2" and the right is titled "Node: h4". Both show the execution of the 'iperf' command for a TCP test. In Node h2, the server is listening on port 80. In Node h4, the client is connecting to 10.0.0.2 on port 80. The results show a bandwidth of approximately 1.17 Mbits/sec, which is slightly above the 1 Mbit/s target, consistent with the lower slice capacity.

```
root@andrea-VirtualBox: /home/andrea/progetto/codice/3# iperf -s -p 80
Server listening on TCP port 80
TCP window size: 85.3 KByte (default)

[ 1] local 10.0.0.2 port 80 connected with 10.0.0.4 port 36236
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-27.3345 sec 2.97 MBytes  912 Kbits/sec
[ 2] local 10.0.0.2 port 80 connected with 10.0.0.4 port 34820
[ ID] Interval      Transfer      Bandwidth
[ 2] 0.0000-31.6161 sec 3.33 MBytes  885 Kbits/sec

root@andrea-VirtualBox: /home/andrea/progetto/codice/3# iperf -c 10.0.0.2 -p 80 -t 10
Client connecting to 10.0.0.2, TCP port 80
TCP window size: 85.3 KByte (default)

[ 1] local 10.0.0.4 port 36236 connected with 10.0.0.2 port 80
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-22.1346 sec 2.97 MBytes  1.13 Mbits/sec

root@andrea-VirtualBox: /home/andrea/progetto/codice/3# iperf -c 10.0.0.2 -p 80
Client connecting to 10.0.0.2, TCP port 80
TCP window size: 85.3 KByte (default)

[ 1] local 10.0.0.4 port 34820 connected with 10.0.0.2 port 80
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-23.9520 sec 3.33 MBytes  1.17 Mbits/sec

root@andrea-VirtualBox: /home/andrea/progetto/codice/3#
```

Nel complesso, i test confermano che il traffico video viene sistematicamente instradato sulla upper slice indipendentemente dalla coppia di host coinvolta, mentre ogni altro tipo di

traffico, sia esso UDP su porte diverse da 9999 oppure TCP, viene instradato sulla lower slice, rispettando così l'obiettivo di prioritizzazione del traffico video richiesto dalla traccia.

Dynamic slicing

Come estensione del problema è stato implementato il Dynamic Slicing, che consiste nell'utilizzare l'upper slice non solo per la trasmissione video ma anche per il traffico normale quando la banda lo consente, garantendo comunque che il traffico video abbia sempre priorità assoluta sul percorso veloce.

Controller

Nella funzione `__init__` sono definiti i seguenti attributi aggiuntivi rispetto al dizionario degli host e alla mappa delle porte:

```
def __init__(self, *args, **kwargs):
    super(DynamicSliceController, self).__init__(*args, **kwargs)

    self.datapaths = {}
    self.monitor_interval = 2
    self.bandwidth_threshold = 1000000 / 8 # 1 Mbps in byte/s

    self.video_stats = {1: 0, 4: 0}
    self.current_speeds = {1: 0.0, 4: 0.0}
    self.global_slice_state = 'UPPER' # Default: Upper slice libera

    self.monitor_thread = hub.spawn(self._monitor)

    self.H = {
        'h1': '00:00:00:00:00:01', 'h2': '00:00:00:00:00:02',
        'h3': '00:00:00:00:00:03', 'h4': '00:00:00:00:00:04'
    }
    self.PORT_MAP = {
        1: {'h1': 1, 'h2': 2, 's2': 3, 's3': 4},
        2: {'s1': 1, 's4': 2},
        3: {'s1': 1, 's4': 2},
        4: {'s2': 1, 's3': 2, 'h3': 3, 'h4': 4}
    }
```

Il dizionario `datapaths` tiene traccia degli switch effettivamente connessi al controller, mentre `video_stats` conserva, per `s1` e `s4`, il numero cumulativo di byte del flusso video letto all'iterazione precedente, necessario per calcolare la velocità istantanea per differenza. Il dizionario `current_speeds` contiene invece la velocità corrente (in byte al secondo) calcolata per ciascuno dei due switch, ed è già inizializzato qui in `__init__` anziché essere creato al bisogno più avanti nel codice. La variabile `global_slice_state` rappresenta lo stato attuale dello slicing per il traffico standard: a differenza di quanto ci si potrebbe aspettare, il valore di partenza è 'UPPER', quindi finché non viene rilevato traffico video sopra soglia, il traffico normale è libero di viaggiare sul percorso veloce.

```
def _monitor(self):
    while True:
        for dp in list(self.datapaths.values()):
            if dp.id in [1, 4]:
                parser = dp.ofproto_parser
                req = parser.OFPFlowStatsRequest(dp)
                dp.send_msg(req)
        hub.sleep(self.monitor_interval)
```

Ogni 2 secondi il thread richiede le statistiche dei flussi solo agli switch s1 e s4, gli unici switch dove transita il traffico diretto verso l'esterno del proprio lato di rete.

```
@set_ev_cls(ofp_event.EventOFPFlowStatsReply, MAIN_DISPATCHER)
def _flow_stats_reply_handler(self, ev):
    body = ev.msg.body
    dpid = ev.msg.datapath.id

    if dpid not in [1, 4]:
        return

    current_video_bytes = 0
    for flow in body:
        if flow.priority == 300:
            if 'udp_dst' in flow.match and flow.match['udp_dst'] == 9999:
                current_video_bytes += flow.byte_count

    prev_bytes = self.video_stats[dpid]
    if prev_bytes == 0:
        delta_bytes = 0
    else:
        delta_bytes = max(0, current_video_bytes - prev_bytes)

    self.video_stats[dpid] = current_video_bytes
    self.current_speeds[dpid] = delta_bytes / self.monitor_interval

    max_video_speed = max(self.current_speeds[1], self.current_speeds[4])
    is_congested = max_video_speed > self.bandwidth_threshold

    if is_congested and self.global_slice_state == 'UPPER':
        self.logger.info(f"*** VIDEO RILEVATO ({max_video_speed*8/1e6:.2f} Mbps). Traffico Standard -> LOWER.")
        self.apply_slice_policy('LOWER')
    elif not is_congested and self.global_slice_state == 'LOWER':
        if max_video_speed < (self.bandwidth_threshold / 2):
            self.logger.info(f"*** VIDEO TERMINATO ({max_video_speed*8/1e6:.2f} Mbps). Traffico Standard -> UPPER.")
            self.apply_slice_policy('UPPER')
```

La vera logica decisionale si trova in `_flow_stats_reply_handler`, agganciata all'evento `EventOFPFlowStatsReply`. La funzione scarta subito la risposta se non proviene da s1 o s4, poi scorre tutti i flussi restituiti dallo switch e somma i byte di quelli con priorità 300 il cui match contiene `udp_dst == 9999`, cioè esattamente le regole che instradano il traffico video verso l'upper slice. Il totale ottenuto, `current_video_bytes`, viene confrontato con il valore precedente salvato in `video_stats[dpid]`: se quest'ultimo era zero il delta viene forzato a zero per evitare un picco fittizio al primo campionamento, altrimenti si calcola `current_video_bytes - prev_bytes`, con un controllo aggiuntivo che azzerava il delta se risultasse negativo (per gestire eventuali reset dei contatori dello switch). Dividendo il delta per l'intervallo di monitoraggio si ottiene la velocità istantanea del video su quello switch, che viene salvata in `current_speeds[dpid]`.

A questo punto la funzione prende il massimo tra le velocità correnti di s1 e s4 e lo confronta con la soglia. Se il valore supera la soglia e lo stato globale è ancora 'UPPER', il controller logga il rilevamento del video e richiama `apply_slice_policy('LOWER')` per spostare il traffico standard sul percorso lento. Se invece il video è sotto soglia, lo stato è 'LOWER' e la velocità massima è scesa sotto la metà della soglia, il controller considera il video terminato e richiama `apply_slice_policy('UPPER')`. Questa soglia dimezzata funge da isteresi: evita che il traffico normale venga spostato ripetutamente avanti e indietro tra i due slice quando la banda video oscilla poco sopra o poco sotto 1 Mbps nelle fasi iniziali o finali di una trasmissione.

Il compito di modificare effettivamente le regole di forwarding è affidato ad `apply_slice_policy`:

```
def apply_slice_policy(self, target_slice):
    self.global_slice_state = target_slice
    for dpid, dp in self.datapaths.items():
        parser = dp.ofproto_parser
        if dpid == 1:
            out_port = self.PORT_MAP[1]['s3'] if target_slice == 'LOWER' else self.PORT_MAP[1]['s2']
            for in_p in [1, 2]:
                for dst in ['h3', 'h4']:
                    match = parser.OFPMatch(in_port=in_p, eth_type=0x0800, eth_dst=self.H[dst])
                    self.add_flow(dp, 250, match, [parser.OFPACTIONOutput(out_port)])
        elif dpid == 4:
            out_port = self.PORT_MAP[4]['s3'] if target_slice == 'LOWER' else self.PORT_MAP[4]['s2']
            for in_p in [3, 4]:
                for dst in ['h1', 'h2']:
                    match = parser.OFPMatch(in_port=in_p, eth_type=0x0800, eth_dst=self.H[dst])
                    self.add_flow(dp, 250, match, [parser.OFPACTIONOutput(out_port)])
```

La funzione riceve come parametro lo slice target ('UPPER' o 'LOWER'), aggiorna subito la variabile di stato globale e poi itera su tutti i datapath conosciuti. Per s1, calcola la porta di uscita da usare guardando la mappa delle porte: se il target è 'LOWER' sceglie la porta verso s3, altrimenti quella verso s2. Fatto questo, reinstalla la regola a priorità 250 per ogni combinazione di porta d'ingresso (1 o 2, cioè h1 o h2) e destinazione (h3 o h4), sovrascrivendo quindi tutte e quattro le regole con la nuova porta di uscita calcolata. Lo stesso ragionamento, speculare, viene applicato a s4 per il traffico diretto verso h1 e h2. In pratica questa funzione non fa altro che riscrivere la regola dinamica di instradamento (quella a priorità 250, distinta dalle regole video a priorità 300 e da quelle locali a priorità 260/290/200) puntandola di volta in volta sull'uplink veloce o su quello lento, a seconda della decisione presa da `_flow_stats_reply_handler`.

Switch (DPID)	Host Destinazione	target_slice	Slice Scelta	Porta di Uscita	Descrizione Azione
S1 (dpid=1)	h3	LOWER	Lower Slice (S3)	4 (s3)	Traffico spostato sul percorso lento (1 Mbps)
S1 (dpid=1)	h3	UPPER	Upper Slice	3 (s2)	

			(S2)		
S1 (dpid=1)	h4	LOWER	Lower Slice (S3)	4 (s3)	Traffico spostato sul percorso lento (1 Mbps)
S1 (dpid=1)	h4	UPPER	Upper Slice (S2)	3 (s2)	Traffico spostato sul percorso veloce (10 Mbps)
S4 (dpid=4)	h1	LOWER	Lower Slice (S3)	2 (s3)	Traffico spostato sul percorso lento (1 Mbps)
S4 (dpid=4)	h1	UPPER	Upper Slice (S2)	1 (s2)	Traffico spostato sul percorso veloce (10 Mbps)
S4 (dpid=4)	h2	LOWER	Lower Slice (S3)	2 (s3)	Traffico spostato sul percorso lento (1 Mbps)
S4 (dpid=4)	h2	UPPER	Upper Slice (S2)	1 (s2)	Traffico spostato sul percorso veloce (10 Mbps)

Chiude il quadro la funzione `_state_change_handler`, che tiene sincronizzato il dizionario `datapaths` con lo stato reale della rete: quando uno switch entra nello stato `MAIN_DISPATCHER` (cioè si è connesso correttamente) viene aggiunto al dizionario se non già presente, mentre quando passa a `DEAD_DISPATCHER` (disconnessione) viene rimosso sia da `datapaths` sia, se presente, da `video_stats`, evitando così che il thread di monitoraggio continui a interrogare uno switch non più raggiungibile o che rimangano dati obsoleti legati a un dispositivo scomparso.

```

@set_ev_cls(ofp_event.EventOFPStateChange, [MAIN_DISPATCHER, DEAD_DISPATCHER])
def _state_change_handler(self, ev):
    datapath = ev.datapath
    if ev.state == MAIN_DISPATCHER:
        if datapath.id not in self.datapaths:
            self.datapaths[datapath.id] = datapath
    elif ev.state == DEAD_DISPATCHER:
        self.datapaths.pop(datapath.id, None)
        self.video_stats.pop(datapath.id, None)

```

Per quanto riguarda le regole installate all'avvio in `switch_features_handler`, la struttura ricalca quella del Service Slicing con qualche affinamento nelle priorità. Su ogni switch resta la regola di default a priorità 0 che dropa qualsiasi pacchetto non altrimenti gestito. La gestione dell'ARP è distribuita in modo che s1 e s4 inoltrino le richieste verso gli host locali e verso il lower slice (escludendo la porta verso s2), mentre s3 fa da semplice ripetitore floodando su tutte le porte tranne quella di ingresso; s2 non ha invece alcuna regola ARP dedicata, per cui un eventuale pacchetto di questo tipo ricadrebbe nella regola di default e verrebbe scartato.

Sul fronte del traffico dati, s1 e s4 seguono la stessa logica speculare. Il traffico UDP sulla porta 9999 in ingresso dagli host locali (priorità 300) viene sempre spedito verso l'uplink corrispondente, indipendentemente dalla destinazione finale: è la regola più specifica e quindi quella con priorità più alta. Subito sopra, a priorità 310, si trovano le regole per il traffico video scambiato localmente tra i due host dello stesso switch (h1↔h2 su s1, h3↔h4 su s4): pur essendo diretto su porta 9999, questo traffico deve prevalere sulla regola generica a priorità 300, altrimenti verrebbe instradato verso l'uplink anche quando la destinazione è locale. Subito sotto 300, a priorità 290, invece si trovano le regole di ritorno dal percorso veloce verso gli host locali. A priorità 260 sono gestite le comunicazioni puramente locali tra i due host collegati allo stesso switch (h1↔h2 su s1, h3↔h4 su s4), che quindi non lasciano mai lo switch. A priorità 250 si inserisce la regola dinamica gestita da `apply_slice_policy`, che copre tutto il resto del traffico IP diretto verso gli host dell'altro lato della rete e la cui porta di uscita cambia nel tempo in base allo stato dello slicing. Infine, a priorità 200, restano le regole di ritorno dal lower slice verso gli host locali. Su s2 e s3 le regole si limitano a inoltrare il traffico ricevuto da un lato verso l'altro (rispettivamente verso s4 o verso s1), senza distinzione di tipo di traffico, dato che la selezione tra i due percorsi avviene a monte, su s1 e s4.

s1

Priorità	Match	Azione	Descrizione
310	IP, UDP, dst_port=9999, src=h1, dst=h2	Output 2	Video locale da h1 a h2, resta confinato allo switch
310	IP, UDP,	Output 1	Video locale da h2 a

	dst_port=9999, src=h2, dst=h1		h1, resta confinato allo switch
300	IP, UDP, dst_port=9999, in_port=1 (h1)	Output 3 (s2)	Video uplink da h1, sempre su upper slice
300	IP, UDP, dst_port=9999, in_port=2 (h2)	Output 3 (s2)	Video uplink da h2, sempre su upper slice
290	IP, in_port=3 (da s2), dst=h1	Output 1	Ritorno dall'upper slice verso h1
290	IP, in_port=3 (da s2), dst=h2	Output 2	Ritorno dall'upper slice verso h2
260	IP, in_port=1, dst=h2	Output 2	Traffico locale h1→h2
260	IP, in_port=2, dst=h1	Output 1	Traffico locale h2→h1
250	IP, in_port=1/2, dst=h3/h4	Output dinamico (3=s2 se UPPER, 4=s3 se LOWER)	Regola dinamica per traffico non-video
200	IP, in_port=4 (da s3), dst=h1	Output 1	Ritorno dal lower slice verso h1
200	IP, in_port=4 (da s3), dst=h2	Output 2	Ritorno dal lower slice verso h2

s2

Priorità	Match	Azione
300	in_port=1 (da s1), IP	Output 2 (verso s4)
300	in_port=2 (da s4), IP	Output 1 (verso s1)

s3

Priorità	Match	Azione
200	in_port=1 (da s1), IP	Output 2 (verso s4)
200	in_port=2 (da s4), IP	Output 1 (verso s1)

s4

Priorità	Match	Azione	Descrizione
310	IP, UDP, dst_port=9999, src=h3, dst=h4	Output 4	Video locale da h3 a h4, resta confinato allo switch
310	IP, UDP, dst_port=9999, src=h4, dst=h3	Output 3	Video locale da h4 a h3, resta confinato allo switch
300	IP, UDP, dst_port=9999, in_port=3 (h3)	Output 1 (s2)	Video uplink da h3
300	IP, UDP, dst_port=9999, in_port=4 (h4)	Output 1 (s2)	Video uplink da h4
290	IP, in_port=1 (da s2), dst=h3	Output 3	Ritorno dall'upper slice verso h3
290	IP, in_port=1 (da s2), dst=h4	Output 4	Ritorno dall'upper slice verso h4
260	IP, in_port=3, dst=h4	Output 4	Traffico locale h3→h4
260	IP, in_port=4, dst=h3	Output 3	Traffico locale h4→h3
250	IP, in_port=3/4, dst=h1/h2	Output dinamico (1=s2 se UPPER, 2=s3 se LOWER)	Regola dinamica per traffico non-video
200	IP, in_port=2 (da s3), dst=h3	Output 3	Ritorno dal lower slice verso h3
200	IP, in_port=2 (da s3), dst=h4	Output 4	Ritorno dal lower slice verso h4

testing

La raggiungibilità completa dei dispositivi resta garantita come nei task precedenti. Di seguito sono riportati i quattro test effettuati per verificare il comportamento dinamico del controller.

Test 1 — Traffico locale "video" tra h1 e h2

Su h2 è stato avviato un server UDP sulla porta 9999 (iperf -s -u -p 9999) e su h1 un client verso h2 sulla stessa porta (iperf -c 10.0.0.2 -u -p 9999 -b 20M -t 15), mentre su s1 è stato

eseguito `tcpdump -i s1-eth3 -nn` per monitorare l'uplink verso s2. Nonostante il traffico sia identificato come "video" dalla porta 9999, h2 riceve il flusso a ~20 Mbits/sec con perdite minime (0,75–2%), e il `tcpdump` su s1-eth3 non cattura alcun pacchetto: il traffico resta quindi confinato a s1 (consegna locale) e non attraversa mai l'uplink verso l'upper slice. Di conseguenza questo tipo di traffico non contribuisce mai al conteggio della banda video usato dal thread di monitoraggio, che infatti verifica solo le regole a priorità 300 effettivamente installate su s1 e s4.

The image shows three terminal windows from a VirtualBox environment, each running a different node's commands. The windows are titled "Node: h2", "Node: h1", and "Node: s1 (root)".

- Node: h2**: Shows the execution of `iperf -s -u -p 9999`. It reports a server listening on UDP port 9999. Two test results are shown: one for a connection to 10.0.0.1 port 48033 with a bandwidth of 20.5 Mbits/sec, and another for a connection to 10.0.0.2 port 48479 with a bandwidth of 20.6 Mbits/sec. Both tests show very low loss (0.75% and 0.75% respectively).
- Node: h1**: Shows the execution of `iperf -c 10.0.0.2 -u -p 9999 -b 20M -t 15`. It reports a client connecting to 10.0.0.2, UDP port 9999, and sending 1470 byte datagrams. The test results show a bandwidth of 20.5 Mbits/sec with 0.029 ms jitter and 530/26747 (2%) loss.
- Node: s1 (root)**: Shows the execution of `tcpdump -i s1-eth3 -nn`. The output indicates that the interface is listening on s1-eth3, link-type EN10MB (Ethernet), and the snapshot length is 262144 bytes. No packets are captured.

Test 2 — Canale libero

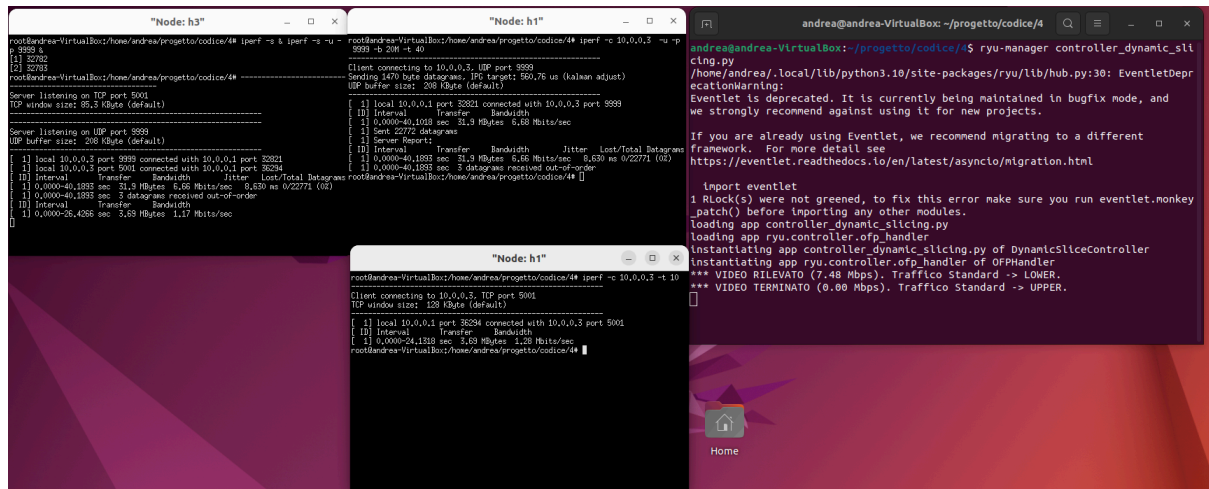
Con h3 in ascolto (`iperf -s`) e h1 come client (`iperf -c 10.0.0.3 -t 10`) su TCP, senza alcun traffico video concorrente, la banda misurata è di circa 8.52 Mbits/sec (8.53 Mbits/sec lato server). Poiché lo stato di default del controller è UPPER, il traffico standard viaggia da subito sul percorso veloce, ottenendo una banda vicina al limite dei 10 Mbps del link.

The image shows two terminal windows from a VirtualBox environment, each running a different node's commands. The windows are titled "Node: h1" and "Node: h3".

- Node: h1**: Shows the execution of `iperf -c 10.0.0.3 -t 10`. It reports a client connecting to 10.0.0.3, TCP port 5001, with a TCP window size of 128 KByte (default). The test results show a bandwidth of 8.52 Mbits/sec with 12.0 MBytes transfer and 11.8200 sec duration.
- Node: h3**: Shows the execution of `iperf -s`. It reports a server listening on TCP port 5001, with a TCP window size of 85.3 KByte (default). The test results show a bandwidth of 8.53 Mbits/sec with 12.0 MBytes transfer and 11.8016 sec duration.

Test 3 — Congestione

Su h3 sono stati avviati contemporaneamente un server TCP (iperf -s) e un server UDP (iperf -s -u -p 9999). Su h1 è stato lanciato un flusso video verso h3 (iperf -c 10.0.0.3 -u -p 9999 -b 20M -t 40), che si è attestato su circa 6.66–7.48 Mb/s/sec, ben oltre la soglia di 1 Mbps. In parallelo, un traffico TCP standard h1→h3 ha ottenuto solo 1.28 Mb/s/sec (e successivamente 1.17 Mb/s/sec su una finestra più lunga), coerente con il limite del lower slice. Il log del controller conferma la transizione



The screenshot shows a terminal window with three panes. The left pane shows the output of 'iperf -s' on Node h3, indicating it is listening on TCP port 5001 and UDP port 9999. The middle pane shows the output of 'iperf -c' on Node h1, showing a video stream to h3 at approximately 7.48 Mb/s and a TCP stream at 1.28 Mb/s. The right pane shows the output of 'ryu-manager controller_dynamic_slicing.py', which displays a warning about Eventlet being deprecated and a log showing the transition from 'VIDEO RILEVATO (7.48 Mbps)' to 'VIDEO TERMINATO (0.00 Mbps)'.

```
root@andrea-VirtualBox:/home/andrea/progetto/codice/4# iperf -s & iperf -s -u -p 9999 &
[1] 32782
[2] 32783
root@andrea-VirtualBox:/home/andrea/progetto/codice/4#
Server listening on TCP port 5001
TCP window size: 65.3 KByte (default)
Server listening on UDP port 9999
UDP buffer size: 208 KByte (default)

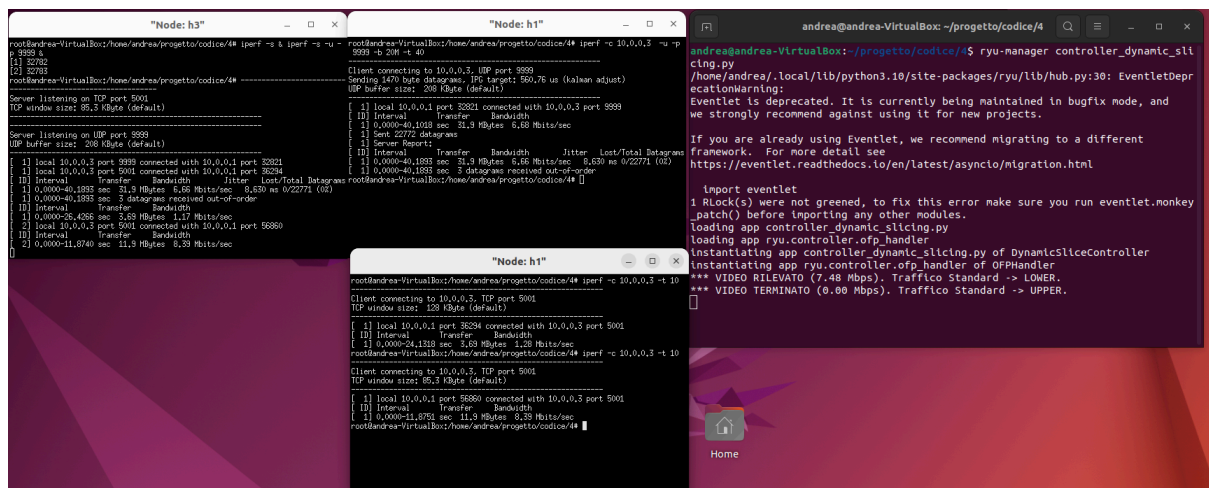
[1] local 10.0.0.3 port 9999 connected with 10.0.0.1 port 33201
[2] local 10.0.0.3 port 5001 connected with 10.0.0.1 port 36294
[0] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[1] 0.0000-40.1028 sec 31.9 MBytes 6.68 Mb/s/sec 0.630 ns 0/22771 (0%)
[1] 0.0000-40.1893 sec 31.9 MBytes 6.68 Mb/s/sec 0.630 ns 0/22771 (0%)
[1] 0.0000-40.1893 sec 3 datagrams received out-of-order
[0] Interval Transfer Bandwidth
[1] 0.0000-26.4266 sec 3.69 MBytes 1.17 Mb/s/sec
[0]

root@andrea-VirtualBox:/home/andrea/progetto/codice/4# iperf -c 10.0.0.3 -u -p 9999 -b 20M -t 40
Client connecting to 10.0.0.3, UDP port 9999
Sending 1470 byte datagrams, IP target: 500.76 us (kcalan adjust)
UDP buffer size: 208 KByte (default)
[1] local 10.0.0.1 port 33201 connected with 10.0.0.3 port 9999
[0] Interval Transfer Bandwidth
[1] 0.0000-40.1028 sec 31.9 MBytes 6.68 Mb/s/sec
[1] Sent 22772 datagrams
[1] Server Report:
[0] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[1] 0.0000-40.1893 sec 31.9 MBytes 6.68 Mb/s/sec 0.630 ns 0/22771 (0%)
[1] 0.0000-40.1893 sec 3 datagrams received out-of-order
root@andrea-VirtualBox:/home/andrea/progetto/codice/4#

andrea@andrea-VirtualBox:~/progetto/codice/4$ ryu-manager controller_dynamic_slicing.py
/home/andrea/.local/lib/python3.10/site-packages/ryu/lib/hub.py:30: EventletDeprecationWarning:
Eventlet is deprecated. It is currently being maintained in bugfix mode, and we strongly recommend against using it for new projects.
If you are already using Eventlet, we recommend migrating to a different framework. For more detail see
https://eventlet.readthedocs.io/en/latest/asyncio/migration.html

Import eventlet
1 RLock(s) were not greened, to fix this error make sure you run eventlet.monkey_patch() before importing any other modules.
loading app controller_dynamic_slicing.py
loading app ryu.controller.ofp_handler
instantiating app controller_dynamic_slicing.py of DynamicSliceController
instantiating app ryu.controller.ofp_handler of OFPHandler
*** VIDEO RILEVATO (7.48 Mbps). Traffico Standard -> LOWER.
*** VIDEO TERMINATO (0.00 Mbps). Traffico Standard -> UPPER.
```

Terminato il flusso video da 40 secondi, un nuovo test TCP h1→h3 (iperf -c 10.0.0.3 -t 10) ha ottenuto 8.39 Mb/s/sec, valore paragonabile al test a canale libero, a conferma del ritorno del traffico standard sull'upper slice.



This screenshot is similar to the previous one, but the video stream on h1 has ended. The 'iperf -c' output on h1 now shows a TCP stream to h3 at 8.39 Mb/s. The controller log on the right shows the transition from 'VIDEO TERMINATO (0.00 Mbps)' to 'Traffico Standard -> UPPER'.

```
root@andrea-VirtualBox:/home/andrea/progetto/codice/4# iperf -s & iperf -s -u -p 9999 &
[1] 32782
[2] 32783
root@andrea-VirtualBox:/home/andrea/progetto/codice/4#
Server listening on TCP port 5001
TCP window size: 65.3 KByte (default)
Server listening on UDP port 9999
UDP buffer size: 208 KByte (default)

[1] local 10.0.0.3 port 9999 connected with 10.0.0.1 port 33201
[2] local 10.0.0.3 port 5001 connected with 10.0.0.1 port 36294
[0] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[1] 0.0000-40.1028 sec 31.9 MBytes 6.68 Mb/s/sec 0.630 ns 0/22771 (0%)
[1] 0.0000-40.1893 sec 31.9 MBytes 6.68 Mb/s/sec 0.630 ns 0/22771 (0%)
[1] 0.0000-40.1893 sec 3 datagrams received out-of-order
[0] Interval Transfer Bandwidth
[1] 0.0000-26.4266 sec 3.69 MBytes 1.17 Mb/s/sec
[0] local 10.0.0.3 port 5001 connected with 10.0.0.1 port 36294
[0] Interval Transfer Bandwidth
[1] 0.0000-11.8740 sec 11.9 MBytes 8.39 Mb/s/sec
[0]

root@andrea-VirtualBox:/home/andrea/progetto/codice/4# iperf -c 10.0.0.3 -t 10
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 128 KByte (default)
[1] local 10.0.0.1 port 36294 connected with 10.0.0.3 port 5001
[0] Interval Transfer Bandwidth
[1] 0.0000-24.1310 sec 3.69 MBytes 1.28 Mb/s/sec
root@andrea-VirtualBox:/home/andrea/progetto/codice/4# iperf -c 10.0.0.3 -t 10
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 65.3 KByte (default)
[1] local 10.0.0.1 port 56889 connected with 10.0.0.3 port 5001
[0] Interval Transfer Bandwidth
[1] 0.0000-11.8751 sec 11.9 MBytes 8.39 Mb/s/sec
root@andrea-VirtualBox:/home/andrea/progetto/codice/4#

andrea@andrea-VirtualBox:~/progetto/codice/4$ ryu-manager controller_dynamic_slicing.py
/home/andrea/.local/lib/python3.10/site-packages/ryu/lib/hub.py:30: EventletDeprecationWarning:
Eventlet is deprecated. It is currently being maintained in bugfix mode, and we strongly recommend against using it for new projects.
If you are already using Eventlet, we recommend migrating to a different framework. For more detail see
https://eventlet.readthedocs.io/en/latest/asyncio/migration.html

Import eventlet
1 RLock(s) were not greened, to fix this error make sure you run eventlet.monkey_patch() before importing any other modules.
loading app controller_dynamic_slicing.py
loading app ryu.controller.ofp_handler
instantiating app controller_dynamic_slicing.py of DynamicSliceController
instantiating app ryu.controller.ofp_handler of OFPHandler
*** VIDEO RILEVATO (7.48 Mbps). Traffico Standard -> LOWER.
*** VIDEO TERMINATO (0.00 Mbps). Traffico Standard -> UPPER.
```

In sintesi, i test confermano che: il traffico video ha sempre priorità sull'upper slice; il traffico standard viene dinamicamente spostato sul lower slice solo quando il video supera 1 Mbps e torna sull'upper slice non appena il video scende sotto la metà della soglia; il traffico locale (h1↔h2, h3↔h4) resta sempre confinato al proprio switch, anche quando è etichettato come "video" dalla porta di destinazione.